

ENSEMBLE LEARNING

Complete Assignment Answers

PART A – THEORETICAL QUESTIONS

Question 1: Can we use Bagging for Regression Problems?

Answer:

Yes, **Bagging can be used for regression problems.**

For regression, Bagging trains multiple base regression models on different bootstrap samples of the training data. The predictions from all models are then combined, usually by taking their **average**.

Working:

1. Create multiple bootstrap samples.
2. Train a regression model on each sample.
3. Generate predictions from all models.
4. Average the predictions.

Formula:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B \hat{y}_b$$

$$\frac{1}{B} \sum_{b=1}^B \hat{y}_b$$

Where:

- B = number of models
- $\hat{y}_b$ = prediction of model b

Example:

```
from sklearn.ensemble import BaggingRegressor

from sklearn.tree import DecisionTreeRegressor

model = BaggingRegressor(

    estimator=DecisionTreeRegressor(),

    n_estimators=100,

    random_state=42

)
```

Easy Definition:

Bagging works for regression by averaging predictions from multiple models.

Question 2: What is the Difference Between Multiple Model Training and Single Model Training?

Answer:

Single Model Training

Only one Machine Learning model is trained on the dataset.

Example:

Dataset → Decision Tree → Prediction

Multiple Model Training

Multiple models are trained, and their predictions are combined.

Example:

┌→ Model 1 ─┐
Dataset ───┼──→ Model 2 ─┼──→ Combined Prediction
└→ Model 3 ─┘

Comparison:

Single Model

Multiple Models

One model	Multiple models
Simpler	More complex
Faster training	Usually more computationally expensive
May have high variance	Often more stable
Example: Decision Tree	Example: Random Forest

Easy Definition:

Single-model learning uses one model; ensemble learning combines multiple models.

Question 3: Explain the Concept of Feature Randomness in Random Forest.

Answer:

Feature randomness means that each Decision Tree in a Random Forest considers only a **random subset of features** when searching for the best split.

This makes individual trees different from each other.

Example:

Suppose the dataset has 10 features.

One tree may consider:

Feature 1, Feature 4, Feature 7

Another tree may consider:

Feature 2, Feature 5, Feature 9

This randomness reduces the correlation between trees.

Benefit:

Less correlation between trees generally improves the diversity of the ensemble and can reduce overfitting compared with a single highly flexible tree.

Easy Definition:

Random Forest randomly selects features for tree splits so that different trees learn different patterns.

Question 4: What is OOB (Out-of-Bag) Score?

Answer:

OOB (Out-of-Bag) Score is an evaluation method used with bootstrap-based ensemble models such as Random Forest.

When bootstrap sampling is performed, some training observations are not selected for a particular tree. These observations are called **Out-of-Bag samples** for that tree.

The OOB samples can be used to estimate model performance without creating a separate validation set.

Example:

If a bootstrap sample contains:

80% → selected for training a tree

20% → OOB for that tree

The OOB observations are used to test that tree.

Python:

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(
```

```
    n_estimators=100,
```

```
    oob_score=True,
```

```
    bootstrap=True,
```

```
    random_state=42
```

```
)
```

```
model.fit(X_train, y_train)
```

```
print(model.oob_score_)
```

Easy Definition:

OOB Score estimates model performance using observations left out of each bootstrap sample.

Question 5: How Can You Measure the Importance of Features in a Random Forest Model?

Answer:

Random Forest provides feature importance scores through:

```
model.feature_importances_
```

These scores indicate how much each feature contributes to reducing impurity across the trees.

Example:

```
import pandas as pd
```

```
importance = pd.DataFrame({
```

```
    "Feature": X.columns,

    "Importance": model.feature_importances_

})

importance = importance.sort_values(

    "Importance",

    ascending=False

)

print(importance)
```

Interpretation:

A higher score means the feature contributed more to the model's splitting decisions under this importance measure.

Important Note:

Impurity-based feature importance can be biased for some types of features. Permutation importance can be used as an alternative.

Easy Definition:

Feature importance tells us which features contribute most to the Random Forest's predictions.

Question 6: Explain the Working Principle of a Bagging Classifier.

Answer:

Bagging Classifier means Bootstrap Aggregating.

It trains multiple base classifiers using different bootstrap samples of the training data.

Steps:

1. Generate multiple bootstrap samples.
2. Train one classifier on each sample.
3. Obtain predictions from all classifiers.
4. Combine predictions using **majority voting**.

Example:

Suppose 5 Decision Trees predict:

Tree 1 → Class A

Tree 2 → Class A

Tree 3 → Class B

Tree 4 → Class A

Tree 5 → Class B

Final prediction:

Class A

because Class A receives the majority of votes.

Easy Definition:

Bagging trains many models on different bootstrap samples and combines their predictions through voting.

Question 7: How Do You Evaluate a Bagging Classifier's Performance?

Answer:

A Bagging Classifier can be evaluated using several metrics.

Classification Metrics:

1. Accuracy
2. Precision
3. Recall
4. F1-Score
5. ROC-AUC
6. Confusion Matrix

Example:

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(  
  
    y_test,  
  
    y_pred  
  
)  
  
print("Accuracy:", accuracy)
```

For imbalanced datasets, accuracy alone may be misleading, so Precision, Recall, F1, ROC-AUC, or PR-AUC should also be considered.

Easy Definition:

Bagging performance is evaluated using classification metrics such as Accuracy, Precision, Recall and F1-Score.

Question 8: How Does a Bagging Regressor Work?

Answer:

A **Bagging Regressor** trains multiple regression models on different bootstrap samples.

The predictions from the individual regressors are generally averaged to produce the final prediction.

Working:

Bootstrap Sample 1 → Regressor 1 → Prediction 100

Bootstrap Sample 2 → Regressor 2 → Prediction 110

Bootstrap Sample 3 → Regressor 3 → Prediction 105

Final Prediction = Average

= 105

Python:

```
from sklearn.ensemble import BaggingRegressor
```

```
model = BaggingRegressor(
```

```
    n_estimators=100,
```

```
    random_state=42
```

```
)
```

Easy Definition:

Bagging Regressor combines predictions from multiple regression models by averaging them.

Question 9: What is the Main Advantage of Ensemble Techniques?

Answer:

The main advantage of ensemble techniques is that they combine multiple models to produce a prediction that is often **more accurate, stable, and robust** than an individual model.

Major Advantages:

- Improved predictive performance
- Reduced variance
- Better generalization
- Greater robustness
- Reduced sensitivity to individual training samples

Examples:

- Bagging
- Random Forest
- Boosting
- Stacking

Easy Definition:

Ensemble learning combines multiple models to obtain a stronger overall model.

Question 10: What is the Main Challenge of Ensemble Methods?

Answer:

The main challenges are:

1. Higher computational cost.
2. Increased training time.
3. More memory requirements.
4. More complex model interpretation.
5. Hyperparameter tuning can be more difficult.
6. Some ensemble models are less transparent than a single Decision Tree.

Easy Definition:

The main challenge is that ensembles can be more complex and computationally expensive than individual models.

Question 11: Explain the Key Idea Behind Ensemble Techniques.

Answer:

The key idea of ensemble learning is:

Combine multiple models to create a stronger model.

Different models may make different errors. Combining them can reduce the effect of individual errors.

Example:

Model 1 → Prediction A

Model 2 → Prediction A

Model 3 → Prediction B

Model 4 → Prediction A

Final → Prediction A

Easy Definition:

Many models working together can perform better than one model working alone.

Question 12: What is a Random Forest Classifier?

Answer:

Random Forest Classifier is an ensemble Machine Learning algorithm that combines many Decision Trees.

Each tree is trained using:

- Bootstrap samples
- Random subsets of features

The final classification is usually determined by **majority voting** across the trees.

Python:

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(  
  
    n_estimators=100,  
  
    random_state=42  
  
)
```

Advantages:

- Good predictive performance
- Handles non-linear relationships
- Less prone to overfitting than a single unrestricted tree
- Provides feature importance
- Works for multi-class classification

Easy Definition:

Random Forest is a collection of Decision Trees whose predictions are combined to make a stronger classifier.

Question 13: What are the Main Types of Ensemble Techniques?

Answer:

The major ensemble techniques are:

1. Bagging

Models are trained independently on bootstrap samples.

Example:

Random Forest

2. Boosting

Models are trained sequentially, with later models focusing more on previous errors.

Examples:

- AdaBoost
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost

3. Stacking

Different models are combined using another model called a **meta-model**.

Summary:

Technique	Main Idea
Bagging	Parallel models on bootstrap samples
Boosting	Sequential models correcting errors
Stacking	Multiple models + meta-model

Question 14: What is Ensemble Learning in Machine Learning?

Answer:

Ensemble Learning is a Machine Learning technique where multiple models are combined to produce a single final prediction.

The models may be:

- Decision Trees
- SVM
- Logistic Regression
- KNN
- Other Machine Learning algorithms

Main Goal:

Improve:

- Accuracy
- Stability
- Generalization
- Robustness

Easy Definition:

Ensemble Learning combines multiple Machine Learning models to produce a stronger overall prediction.

Question 15: When Should We Avoid Using Ensemble Methods?

Answer:

Ensemble methods may be avoided when:

1. Dataset is extremely small and a simple model is sufficient.
2. Interpretability is the highest priority.
3. Computational resources are limited.
4. Very fast prediction is required.
5. A simpler model already provides adequate performance.
6. The additional complexity does not provide meaningful improvement.

Example:

If a simple Logistic Regression gives the required performance and the business needs highly transparent coefficients, using a complex ensemble may not be necessary.

Question 16: How Does Bagging Help in Reducing Overfitting?

Answer:

Bagging reduces overfitting mainly by **reducing variance**.

It trains multiple models on different bootstrap samples.

Each individual model may overfit differently, but averaging or voting across many models can make the final prediction more stable.

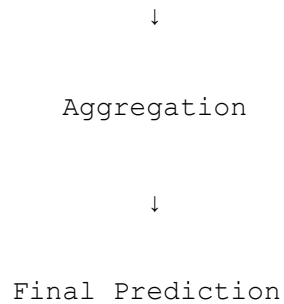
Example:

Tree 1 → Prediction

Tree 2 → Prediction

Tree 3 → Prediction

Tree 4 → Prediction



Easy Definition:

Bagging reduces variance by averaging or voting across multiple models trained on different samples.

Question 17: Why is Random Forest Better Than a Single Decision Tree?

Answer:

Random Forest is often better than a single Decision Tree because it combines many diverse trees.

Random Forest provides:

- Reduced variance
- Better generalization
- More stable predictions
- Less sensitivity to training-data changes
- Feature randomness
- Bootstrap sampling

Comparison:

Decision Tree	Random Forest
One tree	Many trees
Can overfit	Usually more robust
More sensitive to data changes	More stable
Simple to interpret	Less interpretable

Important:

Random Forest is not guaranteed to outperform every single Decision Tree on every dataset.

Question 18: What is the Role of Bootstrap Sampling in Bagging?

Answer:

Bootstrap Sampling creates multiple datasets by randomly sampling observations from the original training dataset **with replacement**.

Each bootstrap sample is used to train a separate model.

Example:

Original:

A B C D E

Bootstrap sample:

A C C D E

Another:

B B C E A

The models therefore see different versions of the training data.

Easy Definition:

Bootstrap sampling creates different training datasets so multiple models can learn different patterns.

Question 19: What are Some Real-World Applications of Ensemble Techniques?

Answer:

Ensemble methods are used in many industries.

Healthcare

- Disease risk prediction
- Patient classification
- Medical diagnosis support

Banking

- Fraud detection
- Credit risk prediction
- Loan default prediction

E-commerce

- Recommendation systems
- Customer churn prediction
- Purchase prediction

Marketing

- Customer segmentation
- Response prediction

Finance

- Risk prediction
- Fraud detection
- Market-related classification tasks

Manufacturing

- Predictive maintenance
- Defect detection

Easy Definition:

Ensemble methods are useful wherever accurate and robust predictions are required.

Question 20: What is the Difference Between Bagging and Boosting?

Answer:

Bagging	Boosting
Models are generally trained independently	Models are trained sequentially
Uses bootstrap samples	Focuses subsequent models on previous errors
Mainly reduces variance	Can reduce bias and variance
Models can be trained in parallel	Sequential dependency limits parallelism
Example: Random Forest	Examples: AdaBoost, Gradient Boosting

Easy Definition:

Bagging trains models independently; Boosting trains models sequentially to correct previous errors.

PART B – PRACTICAL QUESTIONS

Question 21: Train a Bagging Classifier Using Decision Trees on a Sample Dataset and Print Model Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import BaggingClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Base Decision Tree
```

```
base_tree = DecisionTreeClassifier(  
  
    random_state=42  
  
)
```

```
# Bagging Classifier
```

```
model = BaggingClassifier(  
  
    estimator=base_tree,  
  
    n_estimators=100,  
  
    random_state=42  
  
)
```

```
# Train
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(
```

```
        y_test,  
  
        y_pred  
  
)  
  
print("Bagging Classifier Accuracy:", accuracy)
```

Observation:

Bagging combines many Decision Trees and can provide more stable predictions than one tree.

Question 22: Train a Bagging Regressor Using Decision Trees and Evaluate Using MSE.

Python Code:

```
from sklearn.datasets import fetch_california_housing  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.tree import DecisionTreeRegressor  
  
from sklearn.ensemble import BaggingRegressor  
  
from sklearn.metrics import mean_squared_error
```

```
# Load housing dataset
```

```
data = fetch_california_housing()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

```
# Base regressor
```

```
base_model = DecisionTreeRegressor(
```

```
    random_state=42
```

```
)
```

```
# Bagging Regressor

model = BaggingRegressor(

    estimator=base_model,

    n_estimators=100,

    random_state=42

)


# Train

model.fit(X_train, y_train)


# Predict

y_pred = model.predict(X_test)


# MSE

mse = mean_squared_error(

    y_test,

    y_pred

)
```

```
print("Bagging Regressor MSE:", mse)
```

Observation:

Lower MSE indicates better regression performance.

Question 23: Train a Random Forest Classifier on the Breast Cancer Dataset and Print Feature Importance Scores.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

import pandas as pd

# Load dataset

data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Random Forest
```

```
model = RandomForestClassifier(
```

```
    n_estimators=200,
```

```
    random_state=42
```

```
)
```

```
# Train
```

```
model.fit(X_train, y_train)

# Feature importance

importance = pd.DataFrame({

    "Feature": data.feature_names,

    "Importance": model.feature_importances_

})

importance = importance.sort_values(

    "Importance",

    ascending=False

)

print(importance)
```

Observation:

The feature importance table ranks features according to their contribution to impurity reduction in the Random Forest.

Question 24: Train a Random Forest Regressor and Compare Its Performance with a Single Decision Tree.

Python Code:

```
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error


# Load data

data = fetch_california_housing()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(
```

```
        X,

        y,

        test_size=0.2,

        random_state=42

    )


# Single Decision Tree

tree = DecisionTreeRegressor(

    random_state=42

)


tree.fit(X_train, y_train)


tree_pred = tree.predict(X_test)


tree_mse = mean_squared_error(

    y_test,

    tree_pred

)
```

```
# Random Forest

forest = RandomForestRegressor(

    n_estimators=100,

    random_state=42,

    n_jobs=-1

)

forest.fit(X_train, y_train)

forest_pred = forest.predict(X_test)

forest_mse = mean_squared_error(

    y_test,

    forest_pred

)

print("Decision Tree MSE:", tree_mse)

print("Random Forest MSE:", forest_mse)
```

Observation:

The model with the lower MSE performs better on this test set.

Question 25: Compute the Out-of-Bag (OOB) Score for a Random Forest Classifier.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier


# Load data

data = load_breast_cancer()


X = data.data

y = data.target


# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
  
    X,  
  
    y,  
  
    test_size=0.2,  
  
    random_state=42,  
  
    stratify=y  
)
```

```
# Random Forest with OOB
```

```
model = RandomForestClassifier(  
  
    n_estimators=200,  
  
    oob_score=True,  
  
    bootstrap=True,  
  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
print("OOB Score:", model.oob_score_)
```

Observation:

The OOB score provides an internal estimate of generalization performance using samples not selected in each tree's bootstrap sample.

Question 26: Train a Bagging Classifier Using SVM as a Base Estimator and Print Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import make_pipeline

from sklearn.svm import SVC

from sklearn.ensemble import BaggingClassifier

from sklearn.metrics import accuracy_score


# Load data

data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# SVM with scaling
```

```
base_model = make_pipeline(
```

```
    StandardScaler(),
```

```
    SVC(kernel="rbf")
```

```
)
```

```
# Bagging

model = BaggingClassifier(

    estimator=base_model,

    n_estimators=20,

    random_state=42,

    n_jobs=-1

)

model.fit(X_train, y_train)

# Prediction

y_pred = model.predict(X_test)

accuracy = accuracy_score(

    y_test,

    y_pred

)

print("Bagging SVM Accuracy:", accuracy)
```

Question 27: Train a Random Forest Classifier with Different Numbers of Trees and Compare Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score


# Load dataset

data = load_iris()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
  
y,  
  
test_size=0.2,  
  
random_state=42,  
  
stratify=y  
)  
  
tree_values = [10, 50, 100, 200, 500]  
  
for n in tree_values:  
  
    model = RandomForestClassifier(  
  
        n_estimators=n,  
  
        random_state=42  
    )  
  
    model.fit(X_train, y_train)  
  
    y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(  
  
    y_test,  
  
    y_pred  
  
)  
  
print(  
  
    f"Number of Trees: {n}, "  
  
    f"Accuracy: {accuracy:.4f}"  
  
)
```

Observation:

Increasing the number of trees can improve stability, although performance usually reaches a point where additional trees provide diminishing returns.

Question 28: Train a Bagging Classifier Using Logistic Regression as a Base Estimator and Print AUC Score.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import make_pipeline

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import BaggingClassifier

from sklearn.metrics import roc_auc_score


# Load dataset

data = load_breast_cancer()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,
```

```
        random_state=42,

        stratify=y

    )


# Logistic Regression

base_model = make_pipeline(

    StandardScaler(),

    LogisticRegression(max_iter=2000)

)


# Bagging

model = BaggingClassifier(

    estimator=base_model,

    n_estimators=20,

    random_state=42

)


model.fit(X_train, y_train)
```

```
# Probability predictions

y_prob = model.predict_proba(X_test)[:, 1]


# AUC

auc = roc_auc_score(

    y_test,

    y_prob

)


print("AUC Score:", auc)
```

Question 29: Train a Random Forest Regressor and Analyze Feature Importance Scores.

Python Code:

```
from sklearn.datasets import fetch_california_housing

from sklearn.ensemble import RandomForestRegressor

import pandas as pd
```

```
# Load dataset

data = fetch_california_housing()

X = data.data

y = data.target


# Train model

model = RandomForestRegressor(

    n_estimators=200,

    random_state=42,

    n_jobs=-1

)

model.fit(X, y)


# Feature importance

importance = pd.DataFrame({

    "Feature": data.feature_names,
```

```
        "Importance": model.feature_importances_  
    })  
  
importance = importance.sort_values(  
    "Importance",  
    ascending=False  
)  
  
print(importance)
```

Observation:

The feature with the highest importance score contributed most to the forest's impurity reduction.

Question 30: Train an Ensemble Model Using Both Bagging and Random Forest and Compare Accuracy.

Python Code:

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import (

    BaggingClassifier,

    RandomForestClassifier

)

from sklearn.metrics import accuracy_score


# Load dataset

data = load_iris()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,
```

```
        random_state=42,

        stratify=y

    )

# Bagging

bagging = BaggingClassifier(

    estimator=DecisionTreeClassifier(

        random_state=42

    ),

    n_estimators=100,

    random_state=42

)

bagging.fit(X_train, y_train)

bag_pred = bagging.predict(X_test)

bag_accuracy = accuracy_score(

    y_test,
```

```
        bag_pred

    )

# Random Forest

forest = RandomForestClassifier(

    n_estimators=100,

    random_state=42

)

forest.fit(X_train, y_train)

forest_pred = forest.predict(X_test)

forest_accuracy = accuracy_score(

    y_test,

    forest_pred

)

print("Bagging Accuracy:", bag_accuracy)
```

```
print("Random Forest Accuracy:", forest_accuracy)
```

Observation:

Random Forest adds feature randomness to the bagging-style tree ensemble, which can increase diversity among trees.

Question 31: Train a Random Forest Classifier and Tune Hyperparameters Using GridSearchCV.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import (

    train_test_split,

    GridSearchCV

)

from sklearn.ensemble import RandomForestClassifier

# Load dataset

data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Model
```

```
model = RandomForestClassifier(
```

```
    random_state=42
```

```
)
```

```
# Parameter grid
```

```
param_grid = {  
  
    "n_estimators": [50, 100, 200],  
  
    "max_depth": [None, 3, 5, 10],  
  
    "min_samples_split": [2, 5, 10]  
  
}
```

```
# GridSearchCV
```

```
grid = GridSearchCV(  
  
    model,  
  
    param_grid,  
  
    cv=5,  
  
    scoring="accuracy",  
  
    n_jobs=-1  
  
)
```

```
grid.fit(X_train, y_train)
```

```
print("Best Parameters:")
```

```
print(grid.best_params_)
```

```
print("\nBest CV Accuracy:")

print(grid.best_score_)

print("\nTest Accuracy:")

print(grid.score(X_test, y_test))
```

Observation:

GridSearchCV systematically evaluates parameter combinations and selects the best combination according to the chosen cross-validation score.

Question 32: Train a Bagging Regressor with Different Numbers of Base Estimators and Compare Performance.

Python Code:

```
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.ensemble import BaggingRegressor

from sklearn.metrics import mean_squared_error
```

```
# Load dataset
```

```
data = fetch_california_housing()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42

)
```

```
values = [10, 50, 100, 200]
```

```
for n in values:
```

```
    model = BaggingRegressor(  
  
        estimator=DecisionTreeRegressor(  
  
            random_state=42  
  
        ),  
  
        n_estimators=n,  
  
        random_state=42,  
  
        n_jobs=-1  
  
    )
```

```
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)
```

```
    mse = mean_squared_error(  
  
        y_test,  
  
        y_pred  
  
    )
```

```
print(  
  
    f"Estimators: {n}, MSE: {mse:.4f}"  
  
)
```

Observation:

More estimators often make the ensemble more stable, but after enough trees the improvement can become small.

Question 33: Train a Random Forest Classifier and Analyze Misclassified Samples.

Python Code:

```
from sklearn.datasets import load_iris  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.ensemble import RandomForestClassifier  
  
from sklearn.metrics import accuracy_score  
  
# Load dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test, idx_train, idx_test = train_test_split(  
  
    X,  
  
    y,  
  
    range(len(X)),  
  
    test_size=0.2,  
  
    random_state=42,  
  
    stratify=y  
  
)
```

```
# Train
```

```
model = RandomForestClassifier(  
  
    n_estimators=100,  
  
    random_state=42
```

```
)
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:",
```

```
      accuracy_score(y_test, y_pred))
```

```
# Find misclassified samples
```

```
for index, actual, predicted in zip(
```

```
    idx_test,
```

```
    y_test,
```

```
    y_pred
```

```
):
```

```
    if actual != predicted:
```

```
        print(
```

```
            "Sample Index:", index,
```

```
        "| Actual:", data.target_names[actual],  
  
        "| Predicted:", data.target_names[predicted]  
    )
```

Observation:

Misclassified samples can help identify difficult observations or overlapping classes.

Question 34: Train a Bagging Classifier and Compare Its Performance with a Single Decision Tree Classifier.

Python Code:

```
from sklearn.datasets import load_iris  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.tree import DecisionTreeClassifier  
  
from sklearn.ensemble import BaggingClassifier  
  
from sklearn.metrics import accuracy_score  
  
# Load data
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Single Decision Tree
```

```
tree = DecisionTreeClassifier(
```

```
    random_state=42
```

```
)
```

```
tree.fit(X_train, y_train)
```

```
tree_pred = tree.predict(X_test)
```

```
tree_accuracy = accuracy_score(
```

```
    y_test,
```

```
    tree_pred
```

```
)
```

```
# Bagging
```

```
bagging = BaggingClassifier(
```

```
    estimator=DecisionTreeClassifier(
```

```
        random_state=42
```

```
    ),
```

```
    n_estimators=100,
```

```
    random_state=42
```

```
)
```

```
bagging.fit(X_train, y_train)
```

```
bag_pred = bagging.predict(X_test)
```

```
bag_accuracy = accuracy_score(
```

```
    y_test,
```

```
    bag_pred
```

```
)
```

```
print("Decision Tree Accuracy:",
```

```
      tree_accuracy)
```

```
print("Bagging Accuracy:",
```

```
      bag_accuracy)
```

Observation:

Bagging generally reduces variance, but the actual accuracy difference depends on the dataset.

Question 35: Train a Random Forest Classifier and Visualize the Confusion Matrix.

Python Code:

```
import seaborn as sns

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import confusion_matrix


# Load data

data = load_iris()

X = data.data

y = data.target


# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
  
    X,  
  
    y,  
  
    test_size=0.2,  
  
    random_state=42,  
  
    stratify=y  
)
```

```
# Random Forest
```

```
model = RandomForestClassifier(  
  
    n_estimators=100,  
  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
# Confusion matrix

cm = confusion_matrix(

    y_test,

    y_pred

)


# Plot

plt.figure(figsize=(7, 5))


sns.heatmap(

    cm,

    annot=True,

    fmt="d",

    cmap="Blues",

    xticklabels=data.target_names,

    yticklabels=data.target_names

)


plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")

plt.title("Random Forest Confusion Matrix")

plt.show()
```

Question 36: Train a Stacking Classifier Using Decision Trees, SVM, and Logistic Regression and Compare Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import make_pipeline

from sklearn.tree import DecisionTreeClassifier

from sklearn.svm import SVC

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import StackingClassifier
```

```
from sklearn.metrics import accuracy_score

# Load dataset

data = load_iris()

X = data.data

y = data.target

# Split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.2,

    random_state=42,

    stratify=y

)

# Base models

estimators = [
```

```
(  
  
    "decision_tree",  
  
    DecisionTreeClassifier(  
  
        max_depth=4,  
  
        random_state=42  
  
    )  
  
) ,  
  
(  
  
    "svm",  
  
    make_pipeline(  
  
        StandardScaler(),  
  
        SVC(probability=True)  
  
    )  
  
) ,  
  
(  
  
    "logistic",  
  
    make_pipeline(  
  
        StandardScaler(),  
  
        LogisticRegression(  

```

```
        max_iter=2000

    )

)

)

]

# Stacking model

model = StackingClassifier(

    estimators=estimators,

    final_estimator=LogisticRegression(

        max_iter=2000

    )

)

# Train

model.fit(X_train, y_train)

# Predict

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(  
  
    y_test,  
  
    y_pred  
  
)  
  
print("Stacking Classifier Accuracy:",  
  
      accuracy)
```

Observation:

Stacking learns how to combine predictions from different base models using a meta-model.

Question 37: Train a Random Forest Classifier and Print the Top 5 Most Important Features.

Python Code:

```
from sklearn.datasets import load_breast_cancer  
  
from sklearn.ensemble import RandomForestClassifier
```

```
import pandas as pd

# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target

# Train Random Forest

model = RandomForestClassifier(

    n_estimators=200,

    random_state=42

)

model.fit(X, y)

# Feature importance

importance = pd.DataFrame({

    "Feature": data.feature_names,
```

```
        "Importance": model.feature_importances_  
    })  
  
    # Sort  
  
    importance = importance.sort_values(  
  
        "Importance",  
  
        ascending=False  
    )  
  
    # Top 5  
  
    print("Top 5 Important Features:")  
  
    print(importance.head(5))
```

Question 38: Train a Bagging Classifier and Evaluate Performance Using Precision, Recall, and F1-Score.

Python Code:

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import BaggingClassifier
```

```
from sklearn.metrics import (
```

```
    precision_score,
```

```
    recall_score,
```

```
    f1_score
```

```
)
```

```
# Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
        test_size=0.2,

        random_state=42,

        stratify=y

    )

# Bagging

model = BaggingClassifier(

    estimator=DecisionTreeClassifier(

        random_state=42

    ),

    n_estimators=100,

    random_state=42

)

model.fit(X_train, y_train)

# Prediction

y_pred = model.predict(X_test)
```

```
# Metrics

precision = precision_score(

    y_test,

    y_pred

)


recall = recall_score(

    y_test,

    y_pred

)


f1 = f1_score(

    y_test,

    y_pred

)


print("Precision:", precision)

print("Recall:", recall)

print("F1-Score:", f1)
```

Question 39: Train a Random Forest Classifier and Analyze the Effect of max_depth on Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score


# Load data

data = load_iris()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
  
y,  
  
test_size=0.2,  
  
random_state=42,  
  
stratify=y  
)  
  
depths = [1, 2, 3, 5, 10, None]  
  
for depth in depths:  
  
    model = RandomForestClassifier(  
  
        n_estimators=100,  
  
        max_depth=depth,  
  
        random_state=42  
    )  
  
    model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(
```

```
    y_test,
```

```
    y_pred
```

```
)
```

```
print(
```

```
    f"max_depth={depth}, "
```

```
    f"Accuracy={accuracy:.4f}"
```

```
)
```

Observation:

Very small depths can underfit, while allowing greater depth increases model complexity. The best depth should be selected using validation or cross-validation rather than test-set trial and error.

Question 40: Train a Bagging Regressor Using Different Base Estimators (Decision Tree and KNeighbors) and Compare Performance.

Python Code:

```
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor

from sklearn.neighbors import KNeighborsRegressor

from sklearn.ensemble import BaggingRegressor

from sklearn.metrics import mean_squared_error


# Load data

data = fetch_california_housing()

X = data.data

y = data.target


# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
  
    X,  
  
    y,  
  
    test_size=0.2,  
  
    random_state=42  
  
)
```

```
# Bagging with Decision Tree
```

```
tree_bagging = BaggingRegressor(  
  
    estimator=DecisionTreeRegressor(  
  
        random_state=42  
  
    ),  
  
    n_estimators=100,  
  
    random_state=42,  
  
    n_jobs=-1  
  
)
```

```
tree_bagging.fit(  
  
    X_train,
```

```
        y_train

    )

    tree_pred = tree_bagging.predict(

        X_test

    )

    tree_mse = mean_squared_error(

        y_test,

        tree_pred

    )

# Bagging with KNN

knn_bagging = BaggingRegressor(

    estimator=KNeighborsRegressor(

        n_neighbors=5

    ),

    n_estimators=100,

    random_state=42,
```

```
        n_jobs=-1

    )

    knn_bagging.fit(

        X_train,

        y_train

    )

    knn_pred = knn_bagging.predict(

        X_test

    )

    knn_mse = mean_squared_error(

        y_test,

        knn_pred

    )

    print("Decision Tree Bagging MSE:",

        tree_mse)
```

```
print("KNN Bagging MSE:",  
  
      knn_mse)
```

Observation:

The model with the lower MSE performs better on this test set.

Question 41: Train a Random Forest Classifier and Evaluate Its Performance Using ROC-AUC Score.

Python Code:

```
from sklearn.datasets import load_breast_cancer  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.ensemble import RandomForestClassifier  
  
from sklearn.metrics import roc_auc_score  
  
# Load data  
  
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Random Forest
```

```
model = RandomForestClassifier(
```

```
    n_estimators=200,
```

```
    random_state=42
```

```
)
```

```
model.fit(
```

```
        X_train,

        y_train

    )


# Probability

y_prob = model.predict_proba(

    X_test

)[:, 1]


# ROC-AUC

auc = roc_auc_score(

    y_test,

    y_prob

)


print("ROC-AUC Score:", auc)
```

Interpretation:

A ROC-AUC closer to 1 generally indicates better ranking/discrimination between the positive and negative classes.

Question 42: Train a Bagging Classifier and Evaluate Its Performance Using Cross-Validation.

Python Code:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import BaggingClassifier

from sklearn.model_selection import (

    StratifiedKFold,

    cross_val_score

)

# Load dataset

data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Bagging model
```

```
model = BaggingClassifier(
```

```
    estimator=DecisionTreeClassifier(
```

```
        random_state=42
```

```
    ),
```

```
    n_estimators=100,
```

```
    random_state=42
```

```
)
```

```
# Stratified K-Fold
```

```
cv = StratifiedKFold(
```

```
    n_splits=5,
```

```
    shuffle=True,
```

```
    random_state=42
```

```
)
```

```
# Cross-validation

scores = cross_val_score(

    model,

    X,

    y,

    cv=cv,

    scoring="accuracy"

)

print("Fold Accuracies:")

print(scores)

print("\nAverage Accuracy:",

      np.mean(scores))
```

Observation:

Cross-validation provides performance estimates across multiple train-validation splits instead of relying on a single split.

Question 43: Train a Random Forest Classifier and Plot the Precision-Recall Curve.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (

    precision_recall_curve,

    average_precision_score

)

# Load data

data = load_breast_cancer()

X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Random Forest
```

```
model = RandomForestClassifier(
```

```
    n_estimators=200,
```

```
    random_state=42
```

```
)
```

```
model.fit(
```

```
    X_train,
```

```
        y_train

    )

    # Probability scores

    y_prob = model.predict_proba(

        X_test

    )[:, 1]

    # Precision-Recall

    precision, recall, thresholds = precision_recall_curve(

        y_test,

        y_prob

    )

    # Average Precision

    ap = average_precision_score(

        y_test,

        y_prob

    )
```

```
# Plot

plt.figure(figsize=(8, 6))


plt.plot(

    recall,

    precision,

    label=f"Average Precision = {ap:.3f}"

)


plt.xlabel("Recall")

plt.ylabel("Precision")

plt.title("Random Forest Precision-Recall Curve")


plt.legend()

plt.grid()


plt.show()
```

Observation:

The Precision-Recall curve shows the trade-off between precision and recall at different classification thresholds.

Question 44: Train a Stacking Classifier with Random Forest and Logistic Regression and Compare Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import make_pipeline

from sklearn.ensemble import (

    RandomForestClassifier,

    StackingClassifier

)

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Base models
```

```
estimators = [
```

```
    (
```

```

        "random_forest",

        RandomForestClassifier(

            n_estimators=100,

            random_state=42

        )

    ),

    (

        "logistic_regression",

        make_pipeline(

            StandardScaler(),

            LogisticRegression(

                max_iter=2000

            )

        )

    )

]

```

```

# Stacking

```

```

model = StackingClassifier(

```

```
        estimators=estimators,  
  
        final_estimator=LogisticRegression(  
  
            max_iter=2000  
  
        )  
  
    )
```

```
# Train
```

```
model.fit(  
  
    X_train,  
  
    y_train  
  
)
```

```
# Predict
```

```
y_pred = model.predict(  
  
    X_test  
  
)
```

```
accuracy = accuracy_score(  
  
    y_test,
```

```
y_pred

)

print("Stacking Accuracy:", accuracy)
```

Observation:

Stacking combines different model types and learns how to combine their predictions through a final estimator.

Question 45: Train a Bagging Regressor with Different Levels of Bootstrap Samples and Compare Performance.

Answer:

In Bagging, the `max_samples` parameter controls how many training samples are drawn for each bootstrap sample.

We can compare different values of `max_samples`.

Python Code:

```
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.ensemble import BaggingRegressor
```

```
from sklearn.metrics import mean_squared_error
```

```
# Load dataset
```

```
data = fetch_california_housing()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

```
sample_levels = [0.5, 0.7, 1.0]
```

```
for level in sample_levels:

    model = BaggingRegressor(

        estimator=DecisionTreeRegressor(

            random_state=42

        ),

        n_estimators=100,

        max_samples=level,

        bootstrap=True,

        random_state=42,

        n_jobs=-1

    )

    model.fit(

        X_train,

        y_train

    )
```

```

y_pred = model.predict(

    X_test

)

mse = mean_squared_error(

    y_test,

    y_pred

)

print(

    f"max_samples={level}, "

    f"MSE={mse:.4f}"

)

```

Observation:

Changing the bootstrap sample size changes the diversity of the base models. The best value should be selected using validation or cross-validation.

FINAL REVISION TABLE

Topic	Important Point
Ensemble Learning	Combines multiple models
Bagging	Bootstrap Aggregating
Bagging Classifier	Uses voting
Bagging Regressor	Usually averages predictions
Random Forest	Bagging + random feature selection for trees
Bootstrap Sampling	Sampling with replacement
Feature Randomness	Random subset of features considered for splits
OOB Score	Internal estimate using out-of-bag samples
Feature Importance	Measures feature contribution
Stacking	Uses a meta-model to combine base models
Boosting	Sequentially focuses on previous errors
GridSearchCV	Hyperparameter tuning
Cross-Validation	Evaluates model over multiple splits
Precision	Correct positive predictions / predicted positives
Recall	Correct positive predictions / actual positives
F1-Score	Balance of Precision and Recall
ROC-AUC	Measures ranking/discrimination
MSE	Mean squared regression error
Random Forest	Multiple randomized Decision Trees
max_depth	Controls tree depth

n_estimators	Number of trees/base estimators
max_samples	Samples used for each bootstrap model

IMPORTANT FORMULAS

Mean Squared Error

MSE=

1

n

$\sum$

i=1

n

(

y

i

—

y

^

i

)

2

Mean Absolute Error

MAE=

1

n

$$\sum_{i=1}^n |y_i - \hat{y}_i|$$

F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Number of One-vs-One Classifiers

For (K) classes:

$$\frac{K(K-1)}{2}$$

IMPORTANT GOOGLE COLAB NOTE

The practical questions use modern Scikit-learn syntax:

```
BaggingClassifier(estimator=...)
```

and

```
BaggingRegressor(estimator=...)
```

Older tutorials may use:

```
base_estimator=...
```

If an old tutorial gives an error in Google Colab, use the newer `estimator` parameter shown above.

For regression practicals, the examples use the **California Housing dataset** because the old Boston Housing dataset is no longer included in current Scikit-learn releases.

CONCLUSION

Ensemble Learning is a powerful Machine Learning approach that combines multiple models to improve predictive performance and robustness.

Bagging reduces variance by training models on bootstrap samples and combining their predictions.

Random Forest extends this idea by introducing randomness in feature selection, creating diverse Decision Trees.

Stacking combines different types of models through a meta-model, while **Boosting** builds models sequentially to improve previous errors.

The practical programs demonstrate:

- Bagging Classification
 - Bagging Regression
 - Random Forest Classification
 - Random Forest Regression
 - OOB Score
 - Feature Importance
 - Hyperparameter Tuning
 - GridSearchCV
 - Cross-Validation
 - Stacking
 - Confusion Matrix
 - Precision-Recall Curve
 - ROC-AUC
 - Precision, Recall and F1-Score
 - Different bootstrap sample sizes
 - Comparison of ensemble and single models
-